Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Информационная безопасность

Работа 1: разработка защищенного REST API с интеграцией в CI/CD

Выполнил:
Козлов Василий Сергеевич
P3415

Проверила:
Маркина Татьяна Анатольевна

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Стек и организация проекта 4
2 API и воспроизведение запуска 5
3 Защита от SQL Injection и XSS 6
4 Аутентификация и разграничение доступа 7
5 Проверки безопасности в CI 8
6 Функциональные проверки через curl 9
7 Автоматические тесты и SAST 10
8 Результаты SCA и исправление зависимости 11
9 Репозиторий и успешный pipeline 12
10 Разбор предупреждений 13
11 Ответы на контрольные вопросы 14
ЗАКЛЮЧЕНИЕ 15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 16

ВВЕДЕНИЕ

Цель работы - разработать REST API с аутентификацией, защитой от SQL-инъекций и межсайтового скриптинга и встроить автоматические проверки безопасности в процесс разработки.

Объект исследования - серверное приложение для хранения личных заметок. Предмет исследования - проверка прав доступа к данным, безопасная обработка пользовательского ввода, хранение паролей и контроль зависимостей в CI.

Разработаны три метода: вход по логину и паролю, чтение собственных заметок и создание заметки. Для постоянного хранения применяется SQLite. Успешный вход выдает JWT, а все методы с префиксом `/api` используют общий обработчик проверки токена.

Проверка результата состоит из функциональных HTTP-запросов через curl, автоматических тестов негативных сценариев, статического анализа исходного кода и анализа состава зависимостей. Отчеты сохраняются как артефакты GitHub Actions.

Исходный код, README с инструкциями и конфигурация pipeline опубликованы в открытом репозитории: Vaskozlov/cybersec-lab1. Материалы выполнения и исходник настоящего отчета включены в тот же проект.

1 Стек и организация проекта

Выбран JavaScript на Node.js 24 LTS. Express обрабатывает HTTP-запросы, встроенный модуль `node:sqlite` выполняет параметризованные запросы, библиотека `jose` выпускает и проверяет JWT. Для scrypt используется стандартный криптографический модуль Node.js.

Таблица 1 - Основные компоненты проекта

Компонент	**Назначение**
Node.js 24 LTS	Среда CI; локально v26.0.0
Express 5.3.0	Маршрутизация и обработка JSON
SQLite / node:sqlite	Пользователи и личные заметки
jose 6.2.14	JWT / HS256
Helmet 8.3.0	Защитные HTTP-заголовки
express-rate-limit 8.7.2	Ограничение попыток входа
ESLint + security plugin	Статический анализ исходного кода
Dependency-Check 13.0.0	SCA: npm advisories и NVD

`src/app.js` содержит маршруты и middleware; `database.js` - схему SQLite; `passwords.js` - хэширование; `server.js` - конфигурацию и запуск. Команда `scripts/create-user.js` создает пользователя, а `test/api.test.js` проверяет штатные и атакующие запросы. Версии пакетов закреплены в `package.json` и `package-lock.json`.

В базе `users` хранятся ID, имя и хэш пароля. Таблица `notes` содержит ID, ссылку на владельца, заголовок и текст. Внешние ключи включены; таблицы используют строгую типизацию SQLite. Секреты и файл БД исключены из git.

2 API и воспроизведение запуска

Таблица 2 - Методы API

Метод и маршрут	**Доступ**	**Результат**
POST /auth/login	Логин/пароль	200: JWT; 401: отказ
GET /api/data	Bearer JWT	200: до 100 своих заметок
POST /api/data	Bearer JWT	201: созданная заметка

Третьим методом выбран POST `/api/data`: он позволяет проверить запись пользовательского текста в БД и последующее безопасное чтение. Имя поля логина - `username`. Все тела запросов и ответов имеют формат JSON.

Листинг 1 - Установка и запуск из корня репозитория

```
npm ci --ignore-scripts
export JWT_SECRET="$(openssl rand -base64 32)"
# В zsh ввести пароль без отображения:
read -rs 'LAB_PASSWORD?Password: '; echo
export LAB_PASSWORD
npm run create-user -- vasily
unset LAB_PASSWORD
npm start
```

Сервер слушает только 127.0.0.1:3001. Переменная `JWT_SECRET` содержит случайный ключ в base64: минимум 32 байта после декодирования. При отсутствии или недостаточной длине ключа сервер не запускается. Пользователь создается административной командой; пароля по умолчанию нет.

Листинг 2 - Примеры запросов к защищенному методу

```
# Без токена: HTTP 401
curl -i http://127.0.0.1:3001/api/data
# С токеном из POST /auth/login: HTTP 200
curl -i -H "Authorization: Bearer $TOKEN" \
  http://127.0.0.1:3001/api/data
```

3 Защита от SQL Injection и XSS

Значения передаются в SQLite отдельно от SQL через параметры `?`. Строка запроса остается неизменной даже при наличии кавычек или SQL-операторов во входных данных. Конкатенация пользовательского ввода с SQL не используется (листинг 3). Такой подход соответствует рекомендациям OWASP [1].

Листинг 3 - Параметризованные запросы

```
db.prepare(
  'SELECT id, password_hash FROM users WHERE username = ?'
).get(username);

db.prepare(
  'INSERT INTO notes (owner_id, title, body) VALUES (?, ?, ?)'
).run(userId, title, body);
```

Запрос входа с именем `' OR 1=1 --` получил HTTP 401. Дополнительный тест сохраняет SQL-подобный текст как заметку и проверяет, что таблица остается доступной. Защита не основана на черном списке подстрок.

По требованию задания строки заметок экранируются на выходе: символы &, <, >, кавычки и апостроф заменяются HTML-сущностями. Исходный текст сохраняется в БД без экранирования; это предотвращает повторное кодирование при чтении. Реальный результат curl приведен в листинге 4.

Листинг 4 - Входной XSS-текст и поле ответа HTTP 201

```
request.body:  <script>alert(1)</script>
response.body: &lt;script&gt;alert(1)&lt;/script&gt;
```

API возвращает JSON с `nosniff`. Экранирование предназначено для HTML-текста, а не для произвольных URL, скриптов или атрибутов. На клиенте следует применять безопасные средства вывода, например `textContent`, и учитывать контекст [2].

4 Аутентификация и разграничение доступа

Пароль обрабатывается scrypt с N=32768, r=8, p=3 - одним из вариантов параметров OWASP [3]. Для каждого пароля генерируется соль длиной 16 байт. Хэш длиной 64 байта и параметры сохраняются в `password_hash`. При проверке используется сравнение `timingSafeEqual`.

Вход с неверным паролем и неизвестным именем возвращает одинаковую ошибку HTTP 401. Для неизвестного имени также выполняется scrypt с фиктивным хэшем. Число попыток входа ограничено пятью в минуту на IP; превышение дает HTTP 429.

Успешный вход формирует JWT с алгоритмом HS256 и сроком действия 900 секунд. Поля: `sub` - ID пользователя, `iss` - cybersec-lab1, `aud` - cybersec-lab1-client, `iat` и `exp` - время выдачи и истечения, `jti` - случайный идентификатор. Пароль в токен не включается. Подпись обеспечивает целостность, но не шифрует содержимое.

Листинг 5 - Основные параметры проверки JWT

```
await jwtVerify(token, jwtSecret, {
  algorithms: ['HS256'],
  issuer: 'cybersec-lab1',
  audience: 'cybersec-lab1-client',
  typ: 'JWT',
  requiredClaims: ['sub', 'iat', 'exp', 'iss', 'aud', 'jti'],
  maxTokenAge: '900s'
});
```

После проверки подписи и claims сервер проверяет наличие пользователя в БД. Middleware подключен ко всему префиксу `/api`. Клиент не выбирает владельца записи: ID берется из проверенного `sub`. Чтение выполняется с условием `WHERE owner_id = ?`.

Дополнительные меры: предел JSON 16 KiB; ограничения полей заметки; Helmet; `Cache-Control: no-store`; отсутствие стека ошибок в ответах. Для внешнего развертывания требуются HTTPS и настройка reverse proxy.

5 Проверки безопасности в CI

Workflow `.github/workflows/ci.yml` запускается при каждом push, pull request и вручную. Job «API tests and SAST» устанавливает зависимости из lockfile, выполняет HTTP-тесты и ESLint с правилами eslint-plugin-security. Предупреждения также блокируют успешное завершение анализатора.

Job «SCA - OWASP Dependency-Check» выполняет npm audit и Dependency-Check 13.0.0 по `package.json` и `package-lock.json`, включая инструменты разработки. SAST исследует исходный код; SCA сопоставляет компоненты с базами известных уязвимостей. Эти методы дополняют проверку поведения работающего API.

Листинг 6 - Команды проверок в workflow, сокращенно

```
npm ci --ignore-scripts
npm test
npm run sast
npm audit --audit-level=high --json
FEED='https://nvd.nist.gov/feeds/json/cve/2.0/'
dependency-check.sh --project cybersec-lab1 \
  --scan package.json --scan package-lock.json \
  --format HTML --format JSON --out reports \
  --nvdDatafeed "${FEED}nvdcve-2.0-{0}.json.gz" \
  --disableOssIndex --disableAssembly --failOnCVSS 7
```

Dependency-Check использует npm advisory service и официальные NVD JSON 2.0 feeds без API-ключа [4]. База NVD кешируется. Sonatype OSS Index отключен, так как требует отдельной авторизации; проверки NVD и npm сохранены. Порог блокировки - CVSS от 7; ошибки сканера или загрузки также завершают job.

Actions закреплены SHA; архив сканера проверяется SHA-256. Workflow получает только `contents: read`. Результаты тестов, JSON и HTML сохраняются в artifacts, фактические итоги выводятся в Job summary.

6 Функциональные проверки через curl

Скрипт `scripts/capture_http.py` запускает настоящий сервер, создает временную БД и отправляет запросы с помощью curl. Выполнено 10 проверок, ошибок - 0. В публикуемых материалах пароли и JWT заменены пометкой REDACTED; после опыта временная БД удаляется.

Таблица 3 - Фактические результаты запросов

Сценарий	**HTTP**
GET /api/data без токена	401
POST /api/data без токена	401
Правильный логин и пароль	200
Неверный пароль	401
SQL-инъекция в username	401
Некорректный JWT	401
Создание заметки с XSS-текстом	201
Чтение собственной заметки	200
Вход второго пользователя	200
Чтение вторым пользователем: пустой список	200

Листинг 7 - Извлечение из протокола реального ответа

```
GET /api/data (Authorization отсутствует)
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer
Content-Type: application/json; charset=utf-8
{"error":"Authentication required"}

GET /api/data (Bearer JWT второго пользователя)
HTTP/1.1 200 OK
{"data":[]}
```

Полный протокол: `output/evidence/http-transcript.txt`; структурированные результаты: `output/evidence/http-results.json`. Воспроизведение: `python3 scripts/capture_http.py`.

7 Автоматические тесты и SAST

Выполнено 15 автоматических тестов. Проверены выдача JWT, неверный пароль, SQL-инъекции, оба метода без токена, сохранение текста, экранирование XSS, изоляция пользователей, поврежденные и просроченные токены, неправильные алгоритм, issuer и audience, отсутствие exp, alg=none, ограничения JSON, HTTP-заголовки, rate limiter и отказ от короткого ключа.

Результат job: success. ESLint проанализировал 6 файлов: ошибок - 0, предупреждений - 0. Снимок GitHub Actions на рисунке 1 фиксирует фактический результат проверки, а не локально нарисованную имитацию.

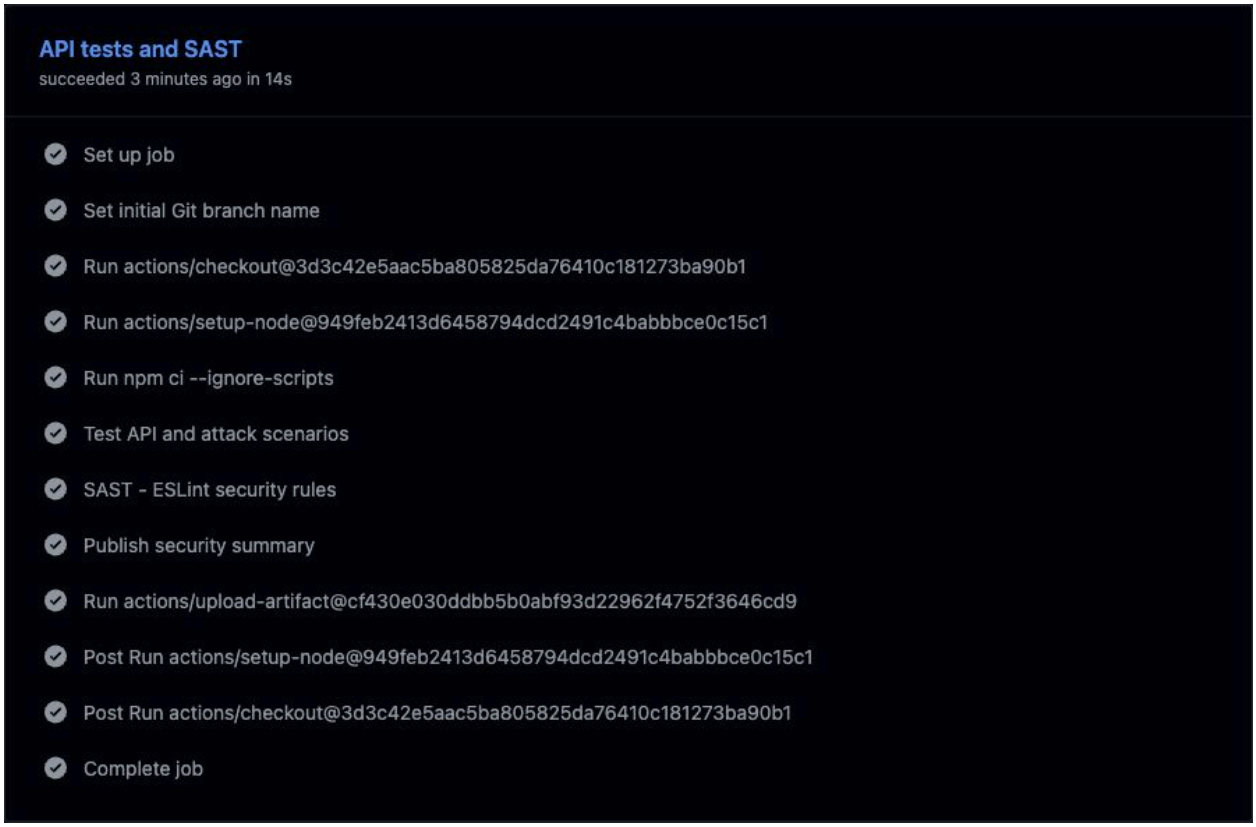


Рисунок 1 - Результаты API tests and SAST в GitHub Actions

Исходные журналы и машиночитаемые отчеты приложены в `output/evidence/`. Пустой список findings означает отсутствие обнаруженных правилом проблем, но не доказывает отсутствие всех возможных уязвимостей.

8 Результаты SCA и исправление зависимости

При первоначальной установке npm audit обнаружил пять moderate-записей, связанных с транзитивным `minimatch` в ESLint. Уведомление GHSA-g3hv-75wg-m249 описывает риск отказа в обслуживании при обработке специальных glob-шаблонов; исправлена версия 11.0.1 [5].

В `package.json` добавлено `overrides.minimatch=11.0.1`, lockfile обновлен. После изменения повторно успешно выполнены статический анализ и тесты. Пакеты не исключались из аудита; подавления CVE для этого исправления не применялись.

Фактический итог npm audit: 0 известных уязвимостей. Dependency-Check: 150 проанализированных компонентов; 0 компонентов с найденными уязвимостями. Результат SCA job: success.

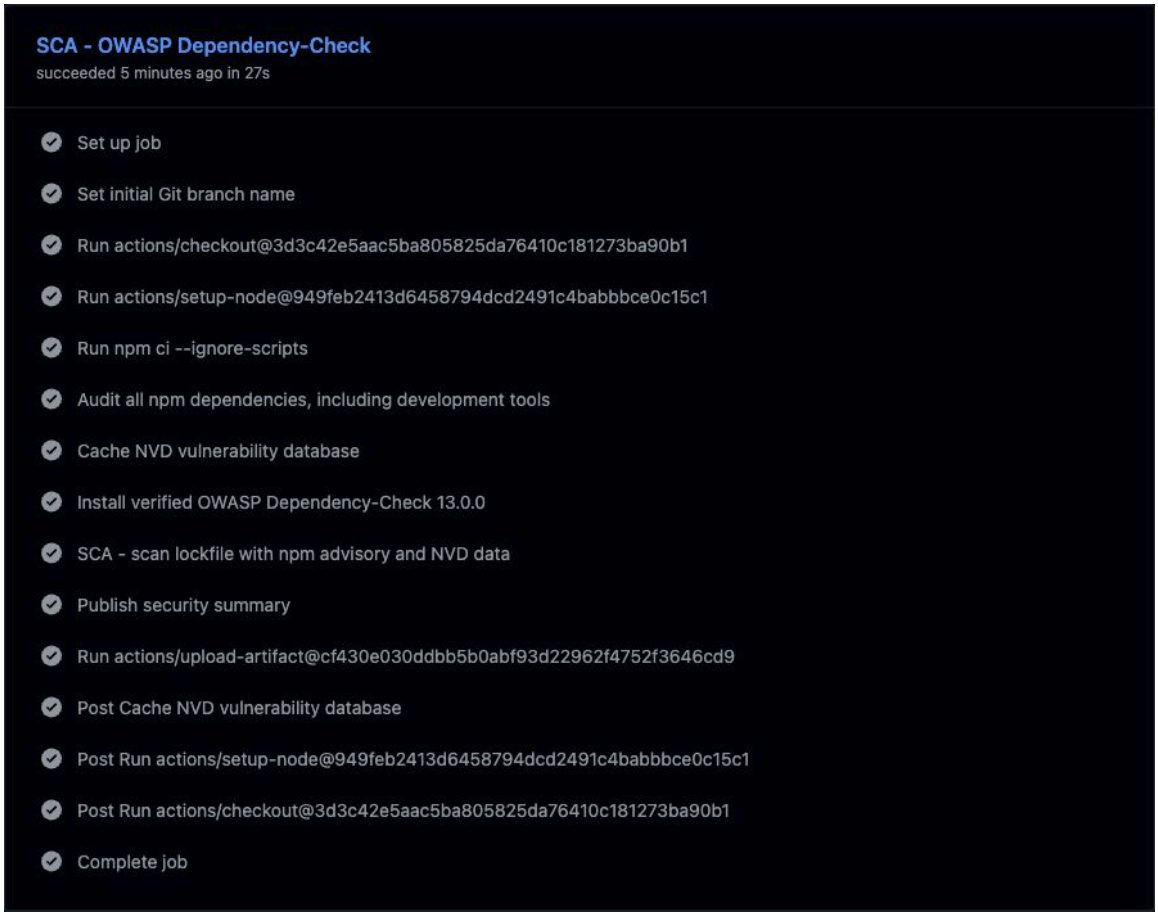


Рисунок 2 - Результаты SCA в GitHub Actions

9 Репозиторий и успешный pipeline

Публичный репозиторий: Vaskozlov/cybersec-lab1. README содержит назначение, описание трех методов, пошаговый запуск, команды curl, объяснение мер защиты и ссылки на отчеты.

Подтвержденный запуск проверок: Security CI, запуск №3. Состояние: success. Проверенный commit: `ed36a64eef5155a126b663452d9d9f716dbb6bdc`.

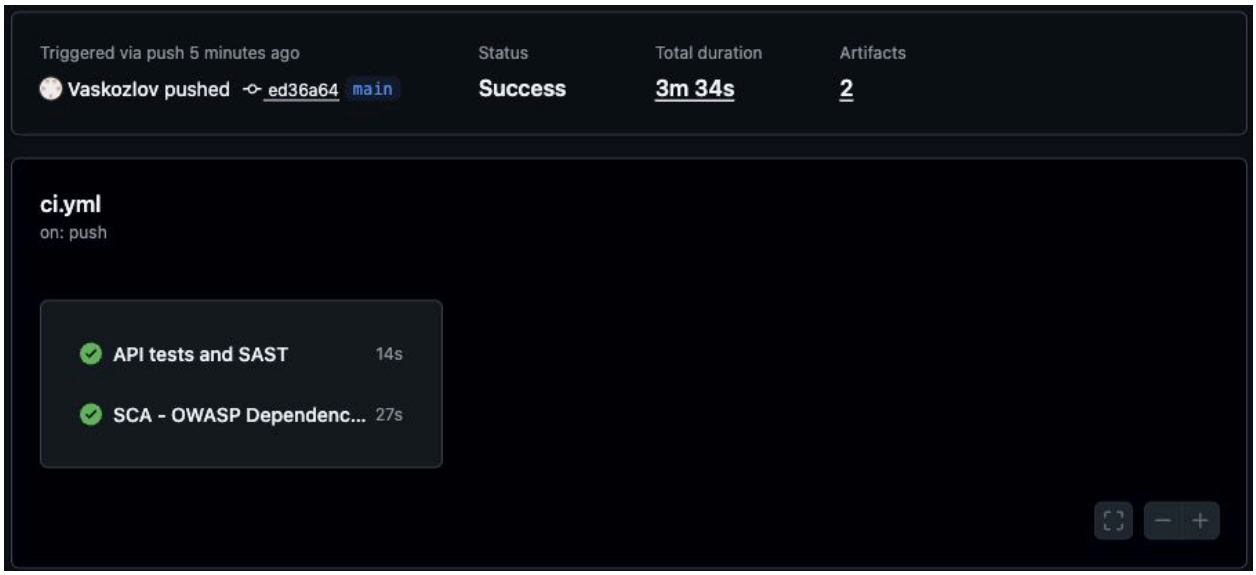


Рисунок 3 - Общий результат pipeline

Таблица 4 - Состав материалов для сдачи

Материал	**Расположение**
Код и зависимости	src/, package.json, package-lock.json
README и workflow	README.md; .github/workflows/ci.yml
HTTP-доказательства	output/evidence/http-transcript.txt
Отчеты сканеров	Actions artifacts и output/evidence/
Скриншоты	output/screenshots/
PDF и исходник	output/pdf/; report.typ

10 Разбор предупреждений

В первом успешном запуске присутствовали предупреждения GitHub Actions об устаревшем runtime Node.js 20 и вызовах `punycode` и `url.parse()` в старых Actions. Это реальные предупреждения инструментов CI, требующие обновления, а не найденные уязвимости кода API.

Actions обновлены с закреплением SHA: checkout 7.0.1, setup-node 7.1.0, upload-artifact 7.0.2 и cache 6.1.0. Их конфигурации используют Node.js 24. После обновления pipeline выполнен повторно. Число предупреждений в журналах итогового запуска: 0.

Подсказка `git init` об имени начальной ветки не является предупреждением безопасности. Она сообщает о настройке Git; checkout затем выбирает требуемую ветку `main`. Эта подсказка не означает пропуск проверки или проблему приложения.

В промежуточном запуске GitHub вывел уведомление уровня notice о миграции `ubuntu-latest` на Ubuntu 26. Оно относится к инфраструктуре, а не к находке безопасности. Для воспроизводимости образ закреплен как `ubuntu-24.04`; начальное имя ветки Git явно установлено в `main` в одноразовом runner. Предупреждения не скрывались настройками подавления сообщений.

Единственное точечное исключение ESLint относится к вызову `appendFileSync` в скрипте формирования summary. Анализатор отмечает нелитеральное имя файла, но путь поступает из `GITHUB_STEP_SUMMARY`, создаваемого runner. Скрипт не обрабатывает HTTP-запросы; пользователь API не может управлять этим путем. Предполагаемый правилом поток недоверенного ввода отсутствует, поэтому предупреждение для данной строки признано ложноположительным и снабжено поясняющим комментарием.

Остальные правила безопасности остаются включенными. Проверка ESLint выполняется с `--max-warnings 0`; ошибки загрузки сканеров и обнаружение зависимостей с CVSS от 7 по-прежнему блокируют pipeline.

11 Ответы на контрольные вопросы

11.1 Хэширование паролей

SHA-256 предназначен для быстрого вычисления: при утечке базы злоумышленник способен быстро перебирать пароли, особенно на GPU. Парольные алгоритмы bcrypt, scrypt и Argon2 используют соль и регулируемую вычислительную стоимость. scrypt дополнительно требует память. Уникальная соль препятствует использованию общих заранее вычисленных таблиц и совпадению хэшей одинаковых паролей разных пользователей.

11.2 Различие SAST и DAST

SAST анализирует исходный код без запуска сервиса, например ищет опасные вызовы и пути обработки ввода. DAST проверяет работающий сервис извне, посылая запросы и оценивая ответы. SCA решает отдельную задачу: проверяет известные уязвимости версий зависимостей. В работе использованы SAST и SCA, а curl и HTTP-тесты подтверждают поведение конкретных сценариев.

11.3 Устройство и проверка JWT

Подписанный JWT состоит из заголовка, payload и подписи в base64url. Заголовок задает тип и алгоритм, payload содержит claims. Сервер проверяет подпись заранее известным ключом и разрешенным алгоритмом, срок действия, issuer и audience. В HS256 ключ общий, поэтому его обладатель способен выпускать токены. Подпись не обеспечивает конфиденциальность payload.

11.4 Риски сторонних библиотек

Уязвимости могут находиться как в прямых, так и в транзитивных зависимостях, включая инструменты CI. Без регулярного аудита известные CVE сохраняются в проекте и могут привести к утечке, обходу доступа или отказу в обслуживании. Lockfile фиксирует версии, но требует повторного сканирования и обновления. Найденная зависимость minimatch показывает практическую роль SCA.

ЗАКЛЮЧЕНИЕ

Разработан REST API с тремя методами, базой SQLite и аутентификацией JWT. SQL-инъекции предотвращаются параметризованными запросами. Строки заметок экранируются при возврате; пароли хранятся в виде соленых scrypt-хэшей. Доступ к заметкам ограничен их владельцем.

Штатное поведение и негативные сценарии подтверждены 15 автоматическими тестами и 10 запросами curl. В CI включены статический анализ и проверка зависимостей через npm audit и OWASP Dependency-Check. Обнаруженная уязвимая dev-зависимость обновлена; результаты представлены реальными журналами, отчетами и скриншотами GitHub Actions.

Учебный стенд рассчитан на один локальный процесс. Развертывание в интернете потребовало бы HTTPS, распределенного ограничения запросов и политики отзыва токенов. Нулевой результат сканирования отражает известные сканерам проблемы на дату проверки, а не абсолютную гарантию безопасности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. OWASP. SQL Injection Prevention Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html (дата обращения: 01.11.2026).

2. OWASP. Cross Site Scripting Prevention Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html (дата обращения: 01.11.2026).

3. OWASP. Password Storage Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата обращения: 01.11.2026).

4. OWASP Dependency-Check. Mirroring External Resources. URL: https://dependency-check.github.io/DependencyCheck/data/mirrornvd.html (дата обращения: 01.11.2026).

5. GitHub Advisory Database. GHSA-g3hv-75wg-m249. URL: https://github.com/advisories/GHSA-g3hv-75wg-m249 (дата обращения: 01.11.2026).

6. Node.js. SQLite API. URL: https://nodejs.org/docs/latest-v24.x/api/sqlite.html (дата обращения: 01.11.2026).

7. jose. Документация и исходный код. URL: https://github.com/panva/jose (дата обращения: 01.11.2026).

8. GitHub Docs. Workflow syntax for GitHub Actions. URL: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax (дата обращения: 01.11.2026).

9. Исходный код лабораторной работы. URL: https://github.com/Vaskozlov/cybersec-lab1 (дата обращения: 01.11.2026).